
PlateformeViewer

Guide d'intégration développeur

Comment contribuer, ajouter un bâtiment et comprendre l'architecture

Auteurs : Oussema Fatnassi, Ali Abakar Issa, Thibault Kine

1. Présentation du projet

PlateformeViewer est un outil de visualisation 3D immersive développé sous Unity 6 (URP). Il permet aux utilisateurs d'explorer le rendu d'un bâtiment réel et de consulter en un clic les plannings de réservation de ses salles, via une connexion à une API REST.

Le projet est conçu pour être modulaire : changer de bâtiment revient à swapper un ScriptableObject de configuration — aucune modification de code n'est nécessaire.

1.1 Stack technique

Technologie	Rôle
Unity 6 (URP)	Moteur 3D, rendu et logique de navigation
C#	Langage de programmation principal
Blender	Modélisation et export du bâtiment 3D
Node.js + React	Front-end du build web (via Unity WebGL)
API REST (JSON)	Source de données live pour les salles

2. Architecture du code

Le projet est organisé en trois couches distinctes :

2.1 Configuration (ScriptableObjects)

Les ScriptableObjects sont des assets Unity qui stockent des données sans être attachés à un GameObject. Ils permettent de configurer le projet sans modifier le code.

Fichier	Rôle
---------	------

BuildingConfigSO.cs	Config principale : URLs API, caméra, liste des salles, mode mock
VisualConfig.cs	Toutes les couleurs et matériaux de la scène
RoomDataSO.cs	Données live d'une salle (une instance par salle). Mis à jour à runtime

2.2 Runtime (MonoBehaviours)

Fichier	Rôle
BuildingManager.cs	Chef d'orchestre : clic raycast, auto-refresh, chargement mock
ApiClient.cs	Singleton HTTP — GET /rooms/{code} avec timeout 10s
RoomController.cs	Composant sur chaque salle — écoute OnDataUpdated
RoomStatusIndicator.cs	Colorie les MeshRenderers selon le statut (libre/occupé/bientôt)
RoomInfoPanel.cs	Sidebar avec slide-in/out — affiche planning complet
RoomSign.cs	Canvas world-space généré procéduralement sur chaque porte
CameraController.cs	Caméra libre WASD + Q/E + right-click look + scroll
SceneStyler.cs	Applique VisualConfig à la caméra et l'éclairage au démarrage
DataLoader.cs	Utilitaire statique pour parser un JSON depuis Resources/

2.3 Outils éditeur (Editor Tools)

Ces scripts ne sont compilés qu'en mode édition Unity (`#if UNITY_EDITOR`). Ils apparaissent dans le menu PlateformeViewer de l'éditeur.

Fichier	Ce que ça fait
RoomLoader.cs	Setup automatique : mappe les JSON MockData → GameObjects, crée les RoomDataSO, assigne les composants
RoomSignSetup.cs	Ajoute un enfant DoorSign sur chaque RoomController
RevertNonCubeMaterials.cs	Remet les matériaux d'origine sur tout sauf les Cube (gérés par le système)

3. Flux de données

Le diagramme ci-dessous décrit le cycle complet, du chargement initial à l'affichage en UI :

Flux de données — vue simplifiée
1. Au démarrage → BuildingManager lit BuildingConfigSO
2a. Si useMockData = true → DataLoader lit Resources/MockData/{roomCode}.json
2b. Sinon → ApiClient.FetchRoom() envoie un GET /rooms/{code}
└ Si API 1 échoue → fallback automatique sur apiBaseUrl2
3. RoomApiResponse reçue → RoomDataSO.UpdateFromResponse()
4. L'événement OnDataUpdated se propage vers :
└ RoomStatusIndicator (couleur du mesh)
└ RoomInfoPanel (sidebar UI)
└ RoomSign (panneau de porte world-space)
5. Auto-refresh toutes les N secondes (refreshInterval dans le SO)

3.1 Format de l'API

L'endpoint attendu est GET {baseUrl}/rooms/{roomCode}. La réponse JSON doit respecter cette structure :

```
{
  "timestamp": "2025-01-23T14:00:00Z",
  "room": {
    "code": "Vert-01",
    "name": "Vert-01",
    "capacity": 20,
    "type": "Salle de travail",
    "status": "occupied",          // "free" | "occupied"
    "current_event": { "title": "...", "start": "...", "end": "...", "organizer": "..." },
  },
  "next_event": { ... },
  "schedule_today": [ { ... }, ... ]
}
```

Note : Le statut "upcoming" n'est pas fourni par l'API — il est calculé côté client dans RoomDataSO.UpdateFromResponse() si le prochain événement commence dans moins de 30 minutes.

4. Ajouter un nouveau bâtiment

C'est la tâche la plus courante pour un nouveau développeur. Voici la procédure complète, étape par étape.

Prérequis

- Le modèle 3D du bâtiment importé dans Unity (depuis Blender, format .fbx ou .glb)
- Les données des salles disponibles (via l'API ou des fichiers JSON mock)
- Unity 6 installé, projet ouvert

Étape 1 — Créer le BuildingConfigSO

Dans le Project panel Unity, clic droit dans Assets/ → Create → PlateformeViewer → Building Config.

Nommez l'asset BuildingConfig_NomDuBatiment.

Champ	Valeur à renseigner
buildingName	Nom lisible du bâtiment (ex : "Tour Gamma")
apiBaseUrl	URL principale de l'API (ex : https://api.tourgamma.fr/v1)
apiBaseUrl2	URL de secours (optionnel)
useMockData	true pendant le développement, false en production
refreshInterval	Fréquence de refresh en secondes (ex : 60)
visualConfig	Assigner le VisualConfig (créer si besoin, voir étape 2)

Étape 2 — Créer ou réutiliser le VisualConfig

Create → PlateformeViewer → Visual Config. Un VisualConfig peut être partagé entre plusieurs bâtiments si le style est identique. Vous pouvez ajuster les couleurs de statut, les matériaux de murs et l'ambiance lumineuse depuis cet asset.

Étape 3 — Préparer les données mock

Créez un fichier JSON par salle dans Assets/Resources/MockData/. Le nom du fichier doit correspondre exactement au roomCode (ex : Vert-01.json).

Structure minimale :

```
{ "timestamp": "2025-01-01T09:00:00Z", "room": { "code": "Vert-01", "name": "Vert-01",  
  "capacity": 20, "type": "Salle de travail", "status": "free",  
  "current_event": null, "next_event": null, "schedule_today": [] } }
```

Étape 4 — Importer et positionner le modèle 3D

1. Glissez le fichier .fbx / .glb dans Assets/Models/.
2. Faites glisser le prefab dans la scène Unity.
3. Chaque pièce doit être un enfant direct du GameObject racine du bâtiment.
4. Le nom de chaque GameObject de salle doit correspondre au roomCode (ex : "Vert-01").
5. Les Cube* enfants de chaque salle recevront le matériau de statut — nommez-les en commençant par "Cube".

Étape 5 — Lancer le setup automatique (RoomLoader)

6. Sélectionnez le GameObject racine du bâtiment dans la scène.
7. Ajoutez le composant RoomLoader.
8. Assignez : building (le GameObject racine), roomLayer (layer "Room"), roomMaterial (matériau de base des salles).
9. Cliquez sur le bouton Setup dans l'Inspector (ou appelez Setup() depuis le menu PlateformeViewer).

Le RoomLoader va automatiquement :

- Faire correspondre chaque GameObject enfant avec son fichier JSON mock
- Créer un RoomDataSO dans Assets/Resources/RoomData/
- Ajouter les composants RoomController et RoomStatusIndicator
- Assigner les matériaux et les colliders sur les Cube*
- Câbler les RoomSign si des DoorSign existent déjà

Étape 6 — Ajouter les panneaux de porte (optionnel)

Menu Unity → PlateformeViewer → Setup Room Signs. Cela crée un enfant DoorSign sur chaque RoomController. Positionnez ensuite chaque DoorSign manuellement dans la vue Scène pour qu'il soit face à la porte.

Étape 7 — Ajouter le bâtiment au BuildingConfigSO

Dans le BuildingConfigSO, dans la liste rooms, ajoutez chaque RoomDataSO créé à l'étape 5. Sans cela, BuildingManager ne saura pas quelles salles refresh.

Étape 8 — Configurer la scène

10. Ajoutez un Manager GameObject vide dans la scène.
 11. Attachez-y les composants : BuildingManager, ApiClient, SceneStyler.
 12. Assignez le BuildingConfigSO à BuildingManager et SceneStyler.
 13. Assignez le VisualConfig à SceneStyler.
 14. Dans BuildingManager, assignez le roomLayerMask au layer "Room".
-

15. Ajoutez un RoomInfoPanel dans le Canvas UI et câblez tous ses champs SerializeField.

Checklist de validation

- ✓ Les noms des GameObjects correspondent aux roomCode des JSON
- ✓ Le layer Room existe dans Unity et est sélectionné dans roomLayerMask
- ✓ Chaque salle a un Collider (ajouté automatiquement par RoomLoader sur les Cube*)
- ✓ Le BuildingConfigSO liste bien tous les RoomDataSO dans rooms
- ✓ useMockData = true pour les tests en local
- ✓ Le prefab bâtiment est dans la scène avec son GameObject racine sélectionné lors du Setup

5. Bonnes pratiques et conventions

5.1 Conventions de nommage

Élément	Convention
roomCode (API)	Format : {Zone}-{Numéro sur 2 chiffres}, ex : Vert-01, Bleu-13
GameObject salle	Doit être identique au roomCode
Enfants Cube	Préfixe "Cube" obligatoire (ex : Cube_Mur, Cube_Sol)
RoomDataSO	Room_{roomCode}.asset (créé automatiquement par RoomLoader)
BuildingConfigSO	BuildingConfig_{NomCourt}.asset
JSON mock	Resources/MockData/{roomCode}.json

5.2 Ne pas modifier les matériaux partagés

RoomStatusIndicator clone les matériaux au démarrage (dans Awake). Ne pas appeler sharedMaterial en runtime — cela modifierait tous les objets utilisant ce matériau dans le projet. Le clonage est automatique, ne pas le contourner.

5.3 Ajouter une nouvelle zone de couleur

Les couleurs de zones (Vert, Bleu, Jaune...) sont définies dans RoomSign.cs dans le tableau statique ZoneColors. Pour ajouter une zone, ajoutez simplement un tuple :

```
("violet", new Color(0.58f, 0.20f, 0.92f)),
```

5.4 Passer en mode production

Dans le BuildingConfigSO, désactivez useMockData et renseignez apiUrl (et apiUrl2 en fallback). Le BuildingManager interrogera automatiquement l'API au démarrage et toutes les refreshInterval secondes.

6. Problèmes connus et solutions

Problème	Solution
Le raycast ne détecte pas les salles	Vérifier que le layer "Room" est bien assigné dans roomLayerMask de BuildingManager et que les Cube* ont ce layer (fait par RoomLoader)
Les couleurs de statut ne changent pas	S'assurer que RoomStatusIndicator est bien câblé dans RoomController. Vérifier que Awake() a été appelé (la scène doit avoir joué au moins une frame)
RoomDataSO introuvable après création	RoomLoader appelle AssetDatabase.SaveAssets() + Refresh() avant de charger l'asset. Si le problème persiste, lancer le Setup une seconde fois
L'API retourne 404 sur /rooms/{code}	Vérifier que le roomCode dans le SO correspond exactement au code attendu par l'API (sensible à la casse)
Le panneau RoomInfoPanel ne glisse pas	Vérifier que _panelWidth est calculé dans Awake() après que le RectTransform soit initialisé (Canvas en mode Screen Space Overlay)

7. Glossaire

Terme	Définition
ScriptableObject (SO)	Asset Unity qui stocke des données sans être attaché à un GameObject. Idéal pour la configuration
MonoBehaviour	Classe de base des composants Unity attachés à des GameObjects
Raycast	Lancer de rayon depuis la caméra vers la scène pour détecter les objets cliqués
Layer Mask	Filtre Unity pour que le raycast n'interagisse qu'avec certains objets (ici : le layer "Room")
URP	Universal Render Pipeline — le pipeline de rendu Unity utilisé dans ce projet
World Space Canvas	Canvas Unity rendu dans l'espace 3D (utilisé pour les RoomSign sur les portes)
Mock data	Données de test locales (fichiers JSON) utilisées à la place de l'API réelle

Pour toute question, consulter le README.md à la racine du projet ou contacter l'équipe via le channel projet.
